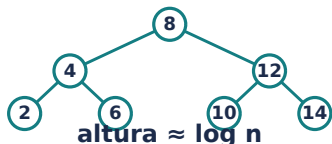


ÁRVORES BALANCEADAS & MULTÍVIAS

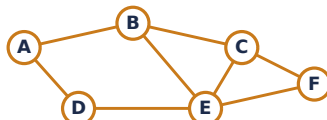


Manter operações eficientes controlando a altura e a organização das chaves na árvore.

- **AVL:** $|\text{altura}(\text{esq}) - \text{altura}(\text{dir})| \leq 1$
- **Rotações:** simples ou duplas restauram o balanceamento
- **Busca/ins./remoção:** $O(\log n)$ em árvore balanceada
- **Red-Black:** balanceamento aproximado com regras de cores
- **B-Tree:** vários filhos/chaves por nó; ideal para disco
- **B+ Tree:** dados nas folhas; folhas encadeadas
- **Trie:** busca por prefixos em $O(m)$, m = tamanho da chave
- **Uso:** índices, sistemas de arquivos e bancos de dados

Dica: Em estruturas balanceadas, o ganho vem de manter a altura próxima de $\log n$. Uma BST comum pode degradar para $O(n)$.

GRAFOS & ALGORITMOS CLÁSSICOS



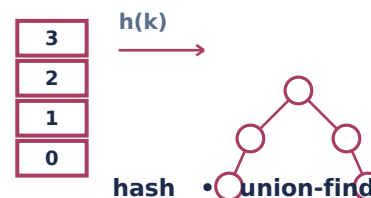
BFS • DFS • menor caminho • MST

Resolver conectividade, caminhos mínimos e árvores geradoras em redes e relações complexas.

- **Representação:** lista de adjacência ou matriz
- **BFS:** $O(V+E)$; menor caminho em grafo não ponderado
- **DFS:** $O(V+E)$; ciclos, componentes e ordenação topológica
- **Dijkstra:** caminhos mínimos com pesos não negativos
- **Bellman-Ford:** aceita pesos negativos e detecta ciclos negativos
- **Floyd-Warshall:** todos os pares em $O(V^3)$
- **Prim/Kruskal:** árvore geradora

Dica: Antes de escolher o algoritmo, observe se há pesos negativos, se o grafo é direcionado e se você precisa de um caminho ou de todos.

HASHING & ESTRUTURAS ESPECIAIS



Usar estruturas especializadas para consultas rápidas, conjuntos dinâmicos e processamento de intervalos.

- **Hashing:** custo médio $O(1)$ para busca/inserção
- **Fator de carga:** $\alpha = n/m$; controla colisões
- **Rehash:** aumenta tabela quando α fica alto
- **Union-Find:** conjuntos disjuntos com find/union
- **Path compression:** encurta caminhos no find
- **Union by rank/size:** evita árvores altas

Dica: Escolha pela operação dominante: hash para chave→valor, Union-Find para conectividade dinâmica e Segment/Fenwick para intervalos.

DP, GREEDY & BACKTRACKING



Escolher estratégias algorítmicas para reduzir recomputação e explorar espaços de busca com eficiência.

- **DP:** subproblemas sobrepostos + subestrutura ótima
- **Memoization:** top-down com cache
- **Tabulation:** bottom-up preenchendo tabela
- **Greedy:** escolhe melhor decisão local a cada passo
- **Backtracking:** tenta, valida e desfaz quando necessário
- **Branch & Bound:** poda ramos usando limites
- **Amortizada:** custo médio de uma sequência de operações
- **Trade-off:** tempo $\times$ memória $\times$ simplicidade da solução

Dica: DP não é “usar tabela” por si só. Primeiro identifique estados, transições, caso base e ordem correta de cálculo.